

OpenCode에서 QE 계산을 안전하게 안내하는 플러그인

Oh My QE는 Quantum ESPRESSO/XSpectra 실습을 slash command로 묶고, 로그인 노드 실행 방지·입력 검증·랩 노트 기록을 기본값으로 강제하는 교육용 OpenCode 플러그인입니다. 이 페이지에는 설치/사용 흐름과 manuscript의 O K-edge convergence test 요약은 함께 정리했습니다.

[/qe command hub](#)[scheduler-first safety](#)[QE input validation](#)[persistent lab notebook](#)[XSpectra convergence notes](#)[PDF download](#)[설치](#)[명령어](#)[실습 흐름](#)[XSpectra 예시](#)[Convergence](#)[수업 운영 메모](#)

1. 설치: OpenCode 위에 QE 실습 레이어를 엮습니다

먼저 OpenCode를 설치한 뒤, Oh My QE 저장소를 받아 설치 스크립트를 실행합니다. 설치 후에는 OpenCode의 `/` command palette에서 `/qe` 계열 명령이 보입니다.

```
$ git clone https://github.com/material-JH/oh-my-qe.git
$ ./oh-my-qe/install.sh
# 필요하면 PATH 추가
$ export PATH="$HOME/.local/bin:$PATH"
$ cd /scratch/$USER/xspectra-tutorial
$ opencode
```

신규 데스크톱 앱 베타 버전 출시 macOS, Windows, Linux 지원. 지금 다운로드

오픈 소스 **AI** 코딩 에이전트

무료 모델이 포함되어 있으며, 어떤 제공자의 모델이든 연결 가능합니다.
Claude, GPT, Gemini 등을 포함합니다.

`curl` `npm` `bun` `brew` `paru`

```
curl -fsSL https://opencode.ai/install | bash
```

OpenCode는 모델 연결이 가능한 터미널 기반 코딩 에이전트입니다.

commands

`~/.config/opencode/commands/qe*.md` 에 slash command를 설치합니다.

plugin hook

`oh-my-qe.js` 가 QE 실행 안전 규칙과 랩 노트 기록을 처리합니다.

validator agent

`qe-input-validator` 가 QE 입력의 BLOCKER/WARN 항목을 읽기 전용으로 검사합니다.

omq helper

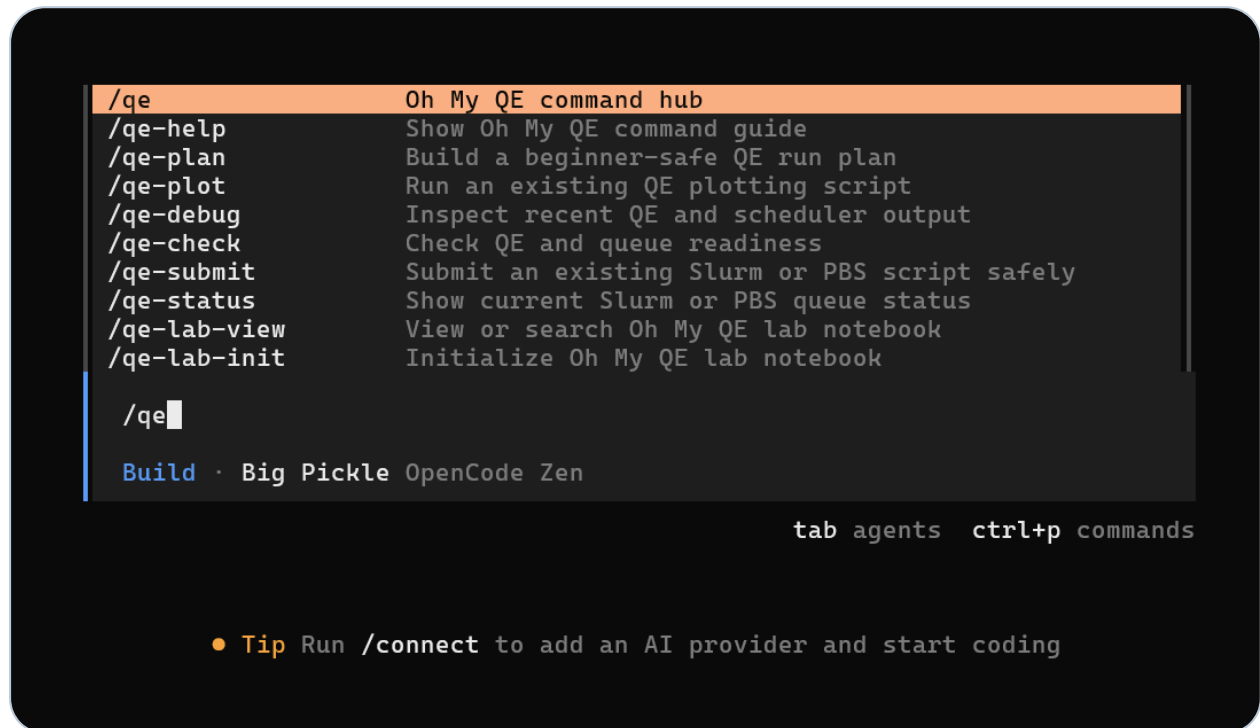
`~/.local/bin/omq` 가 check, submit, plot, lab, validate 작업을 수행합니다.

 material-JH Set Korean as default response language	e1bf470 · 11 hours ago	 17 Commits
 <code>.opencode</code>	Add QE input validator subagent	16 hours ago
 <code>bin</code>	Add QE input validator subagent	16 hours ago
 <code>.gitignore</code>	Initial oh-my-xspectra opencode plugin	yesterday
 <code>AGENTS.md</code>	Set Korean as default response language	11 hours ago
 <code>install.sh</code>	Add QE input validator subagent	16 hours ago
 <code>opencode.json</code>	Initial oh-my-xspectra opencode plugin	yesterday
 <code>uninstall.sh</code>	Add QE input validator subagent	16 hours ago

저장소는 OpenCode commands, plugin, validator agent, shell helper로 구성됩니다.

2. Slash command: 10개 top-level 명령 + /qe hub

OpenCode의 `/` autocomplete가 보여 주는 항목 수가 제한될 수 있으므로, 자주 쓰는 명령은 top-level로 두고 나머지는 `/qe <subcommand>` hub에 넣었습니다.



`/qe` 를 입력하면 QE 실습 명령이 command palette에 나타납니다.

Top-level slash commands

명령	역할
<code>/qe</code>	덜 자주 쓰는 워크플로를 모아 둔 command hub
<code>/qe-help</code>	명령어 안내 출력
<code>/qe-check [DIR]</code>	QE 실행파일, Python, 스케줄러, project 쓰기 권한 확인
<code>/qe-plan [DIR]</code>	현재 파일 기준으로 안전한 실행 계획 작성
<code>/qe-submit <SCRIPT> [DIR]</code>	<code>.sbatch</code> , <code>.slurm</code> , <code>.pbs</code> 스크립트 제출

명령	역할
<code>/qe-status</code>	Slurm/PBS 큐 상태 확인
<code>/qe-debug [DIR]</code>	최근 출력과 scheduler log를 읽고 실패 원인 설명
<code>/qe-plot [DIR]</code>	기존 plotting script 또는 2-column <code>.dat</code> 시각화
<code>/qe-lab-init [DIR]</code>	<code>.oh-my-qe/lab/</code> 랩 노트 생성
<code>/qe-lab-view [DIR]</code> <code>[QUERY]</code>	랩 노트 보기/검색

Hub subcommands

Subcommand	역할
<code>resources</code> <code>[pseudo structure cif all]</code>	검증된 pseudo/구조 DB 링크
<code>files [DIR]</code>	입력, 출력, pseudo, scheduler, plot 파일 목록
<code>env [DIR]</code>	host, scheduler, MPI, QE path, Python, disk 요약
<code>mode [expert beginner]</code>	초보자/전문가 응답 모드 전환
<code>expert <REQUEST></code>	짧은 전문가 모드 답변 요청
<code>explain-input <FILE></code>	QE 입력 파일 설명
<code>validate-input <FILE DIR></code>	QE 입력 정적 검사
<code>defect-plan [DIR]</code>	결함 계산 계획
<code>xspectra-plan [DIR]</code>	XSpectra 계산 계획

Subcommand

역할

lab-update [DIR]

프로젝트 상태에서 랩 노트 synthesis 갱신

Oh My QE 슬래시 명령어 안내

최상위 슬래시 명령어 (/qe 타이핑 시 표시, 최대 10개)

명령어	설명
/qe	덜 자주 쓰는 워크플로를 모아둔 허브
/qe-help	이 안내를 출력
/qe-check [DIR]	QE 실행파일, Python, 스케줄러 준비 상태 확인
/qe-plan [DIR]	안전한 실행 계획 수립 (무거운 작업은 실행 안 함)
/qe-submit <SCRIPT> [DIR]	.sbatch / .slurm / .pbs 스크립트 제출
/qe-status	현재 큐 작업 조회
/qe-debug [DIR]	최근 출력 로그 확인 후 오류 원인 설명
/qe-plot [DIR]	기존 플롯 스크립트 실행
/qe-lab-init [DIR]	마크다운 연구 노트 생성
/qe-lab-view [DIR] [QUERY]	연구 노트 보기/검색

Top-level command 출력 예시.

서브커맨드	설명
resources [pseudo structure cif all]	슈도퍼텐셜/구조 DB 링크 출력
files [DIR]	프로젝트 파일 목록
env [DIR]	환경 요약 (호스트, 스케줄러, MPI, QE, Python, 디스크)
mode [expert beginner]	응답 모드 전환
expert <REQUEST>	전문가 모드로 요청 실행
explain-input <FILE>	QE 입력 파일 설명
validate-input <FILE DIR>	입력 파일 정적 검사 (BLOCKER/WARN)
defect-plan [DIR]	결함 계산 계획 수립
xspectra-plan [DIR]	XSpectra 계산 계획 수립
lab-update [DIR]	연구 노트 갱신

Hub subcommand 출력 예시.

3. 기본 워크플로: 확인 → 계획 → 검증 → 제출 → 기록

- 1 `/qe-check` 로 `pw.x` , `xspectra.x` , Python, queue command, `env.sh` , project 쓰기 권한을 확인합니다.
- 2 `/qe-lab-init` 로 `.oh-my-qe/lab/index.md` , `notes.md` , `log.md` 를 만듭니다. 원시 QE 입력/출력은 source of truth이고, 랩 노트는 유지되는 synthesis layer입니다.
- 3 `/qe-plan` 또는 `/qe xspectra-plan` 으로 무거운 계산을 바로 실행하지 않는 계획을 만듭니다.
- 4 입력 파일을 만들거나 바꾼 뒤 `/qe validate-input` 또는 validator agent 로 BLOCKER/WARN을 확인합니다.
- 5 `/qe-submit` 은 기존 scheduler script만 제출합니다. heavy QE 실행은 Slurm/PBS queue를 거칩니다.
- 6 `/qe-status` , `/qe-debug` , `/qe-plot` , `/qe-lab-view` 로 상태·오류·그림·기록을 다룹니다.

안전 규칙. active allocation(`SLURM_JOB_ID` , `PBS_JOBID` , `PBS_JOB_ID`)이 없으면 `pw.x` , `xspectra.x` 같은 QE executable을 로그인 노드에서 직접 실행하지 않습니다. cluster policy, queue, account, walltime이 불명확하면 값을 만들어 내지 말고 instructor/admin에게 확인합니다.

랩 노트북은 `/home01/e1956a02/.oh-my-qe/lab/`에 위치해 있습니다.

- `index.md` - 중요 랩 페이지 목록 (카탈로그)
- `notes.md` - 현재까지의 종합/해석 노트
- `log.md` - 추가 전용 시계열 기록 (스케줄러/플롯/QE 활동 자동 기록)

원시 QE 입력(`*.in`) 및 출력 파일은 **불변의 진실 공급원(source of truth)**이고, `.oh-my-qe/lab/`은 그 위에 쌓는 **유지보수된 합성 계층(maintained synthesis layer)**입니다. 계산을 제출/분석/플롯할 때마다 랩 노트를 업데이트하게 됩니다.

랩 노트는 계산 제출·plot·debug·QE shell activity를 시간순으로 남기는 장치입니다.

4. XSpectra 튜토리얼에서 쓰는 방식

Diamond C K-edge

처음에는 짧은 diamond 예제로 SCF → XSpectra → replot → core-hole SCF → core-hole XSpectra 흐름을 익힙니다. Oh My QE는 현재 파일을 읽고 어떤 입력과 scheduler script를 써야 하는지 초보자용 순서로 풀어 줍니다.

SrTiO3 O K-edge

흡수 원자를 별도 species(Oh)로 분리하고 xiabs, HCH/FCH pseudo, tot_charge, supercell, k mesh를 확인합니다. 논문 수준 해석 전에는 아래 convergence 기준으로 production setting을 재검토합니다.

1. 확인된 계산 유형 (Diamond C K-edge XANES)

단계	입력 파일	실행 파일	설명
1a	diamond.scf.in	pw.x	SCF (core-hole 없음)
1b	diamond.xspectra.in	xspectra.x	XANES (점유 상태 포함)
1c	diamond.xspectra_replot.in	xspectra.x	XANES 재플롯 (점유 상태 제거)
1d	diamonhd.scf.in	pw.x	SCF (core-hole 있음, tot_charge=+1.0)
1e	diamonhd.xspectra.in	xspectra.x	XANES (core-hole, 점유 상태 제거)

SrTiO3 O K-edge 예제도 함께 제공되어 있습니다.

2. 제출할 스케줄러 스크립트

- Slurm: scheduler/slurm_diamond.sbatch (sbatch scheduler/slurm_diamond.sbatch)
- PBS: scheduler/pbs_diamond.pbs (qsub scheduler/pbs_diamond.pbs)

3. 성공 지표 (출력 파일)

- diamond.scf.out - final ! 확인
- diamond.xspectra.out / diamond.xspectra_replot.out / diamonhd.xspectra.out - JOB DONE 확인
- diamond.xspectra.dat / diamond.xspectra_replot.dat / diamonhd.xspectra.dat - 생성된 스펙트럼

/qe-plan C-K edge diamond 출력 예시.

← Edit .oh-my-qe/lab/log.md

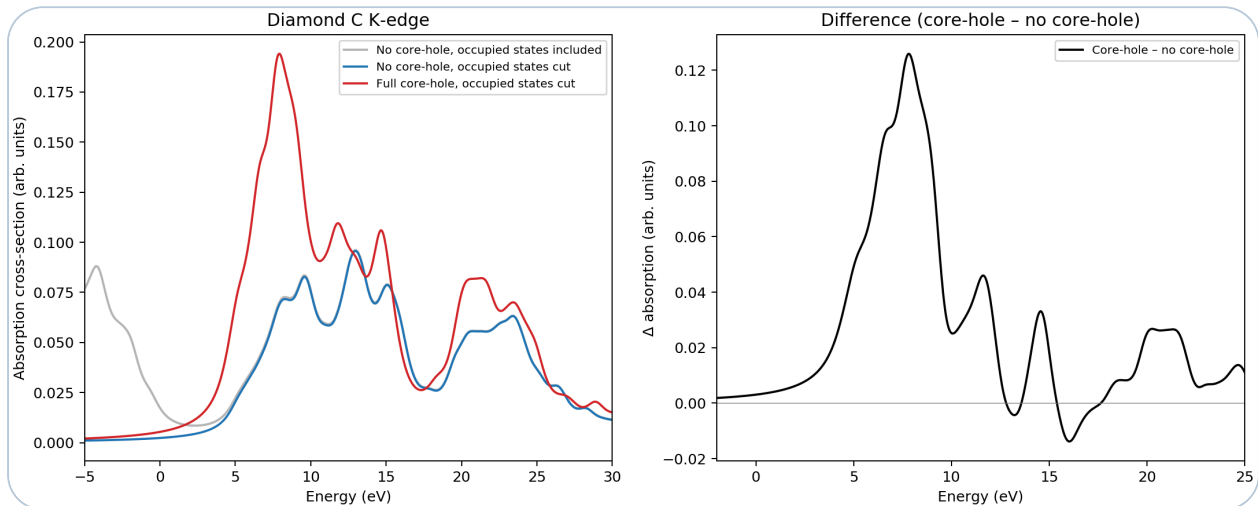
```
8 - Pseudopotentials bundled in `diamond/pseudo/` (C_PBE_TM_2pj.UPF, Ch_PBE_TM_2pj.UPF)
9 - Warning: do not run pw.x/xspectra.x on login node
10 - Lab notebook initialized: index.md, notes.md, log.md created
11
12 + ## 2026-06-25 - Job submitted: Diamond C K-edge XANES
13 +
14 + - Created `env.sh` for this cluster (intel/19.1.2, impi/19.1.2, qe/7.3.1, python/3.7,
    - craype-mic-knl)
15 + - Submitted: `qsub /scratch/e1956a02/xspectra-tutorial/scheduler/pbs_diamond.pbs`
16 + - Job ID: 22584109.pbs (gachon queue, 1 node, 16 MPI procs, 30 min walltime)
17 + - Runs `run_all_examples.sh diamond-only` → 5-step pipeline (SCF no-hole → XSpectra →
    replot → SCF core-hole → XSpectra core-hole)
18 + - Status: QUEUED
19 +
```

완료되었습니다. `qstat`으로 진행 상황을 확인할 수 있습니다:

```
qstat -u $(whoami)
```

작업이 완료되면 `python3 plot_spectra.py` (로그인 노드에서 안전)로 스펙트럼을 시각화합니다.

작업 제출 후 job id와 상태가 랩 노트에 기록됩니다.



실습의 첫 성공 지표는 로그의 `JOB DONE` 과 생성된 spectrum 파일, 그리고 plot입니다.

5. Manuscript convergence test: 빠른 실습값과 production 해석값을 분리합니다

Supplemental Material의 O K-edge HCH convergence test는 wavefunction cutoff, k-point mesh, Lanczos iteration을 독립적으로 바꾸며 $t_{2g}-e_g$ peak spacing ΔE 와 peak-height ratio $I(t_{2g})/I(e_g)$ 를 추적했습니다. 결론은 단순합니다: cutoff와 Lanczos는 안정적이지만, k mesh는 실제 해석 품질을 좌우합니다.

55 Ry

ecutwfc

40/55/70 Ry에서 $\Delta E=5.86$ eV , intensity ratio도 0.780 전후로 안정적입니다.

2000

Lanczos

500/1000/2000 iteration에서 두 지표가 변하지 않아 production 값으로 충분합니다.

8³

Unit-cell k mesh

2³는 부적절하고 4³도 line shape 차이가 납니다. 6³ 이후 안정, production은 8³.

4³

Supercell k mesh

2³ supercell은 under-sampled. 3³부터 주요 feature가 안정되고 4³는 5³ reference에 가깝습니다.

수업용 입력과 논문용 입력은 목적이 다릅니다. 튜토리얼의 작은 mesh는 반나절 실습에서 흐름을 익히기 위한 빠른 설정일 수 있습니다. peak 위치, intensity ratio, 결함 농도 해석을 논문 수준으로 말하려면 아래 production 기준으로 재계산해야 합니다.

Table S6 요약: scalar feature convergence

Parameter	Value	ΔE (eV)	$I(t2g)/I(eg)$	판정
Wavefunction cutoff	40 Ry	5.86	0.782	stable
	55 Ry	5.86	0.780	baseline
	70 Ry	5.86	0.781	stable
Unit-cell k-grid	$2 \times 2 \times 2$	5.71	0.599	inadequate
	$4 \times 4 \times 4$	5.86	0.780	visible line-shape difference
	$6 \times 6 \times 6$	5.81	0.770	mostly stable
	$8 \times 8 \times 8$	5.81	0.765	production
	$10 \times 10 \times 10$	5.81	0.765	stable
	$12 \times 12 \times 12$	5.81	0.765	reference
Lanczos iterations	500	5.86	0.780	stable
	1000	5.86	0.780	stable
	2000	5.86	0.780	production

Table S7 요약: extended k-point line-shape difference

System	Mesh	RMS vs reference	Max abs. vs reference	읽는 법
Unit cell, reference 12 ³	2 ³	0.2141	0.5020	not usable
	4 ³	0.0652	0.1408	training- only speed setting
	6 ³	0.0261	0.0527	acceptable trend
	8 ³	0.0113	0.0301	production choice
	10 ³	0.0043	0.0132	near reference
Pristine 2×2×2 supercell, reference 5 ³	2 ³	0.0508	0.1460	under- sampled
	3 ³	0.0195	0.0700	main features stable
	4 ³	0.0072	0.0286	production choice
$\delta = 0.125$ vacancy supercell, reference 5 ³	2 ³	0.0359	0.1027	under- sampled
	3 ³	0.0143	0.0334	main features stable
	4 ³	0.0040	0.0127	production choice

Method-sensitivity checks

Broadening

$\delta \approx 0.25$ O K-edge에서 $\gamma = 0.8/1.2$ eV broadening을 바꿔도 broad high-post-edge plateau가 생기지 않습니다.

DFT+U

Ti-only Dudarev $U=2, 4$ eV scan은 near-stoichiometric 기준과 high- δ vacancy line shape mismatch를 해결하지 못했습니다.

Empty states

Stoichiometric $\text{nbnd}=30/45/60$, vacancy $\text{nbnd}=240/320/400$ 범위에서 manuscript metric window의 line shape는 안정적입니다.

6. 수업 운영 메모

처음 10분

`/qe-check` 와 `/qe-lab-init` 만 성공시키면 이후 오류 추적이 쉬워집니다. 학생별 환경 차이는 이 단계에서 드러납니다.

계산 제출 전

입력 변경 후에는 `/qe validate-input` 결과의 BLOCKER를 먼저 닫습니다. BLOCKER가 남은 제출은 instructor override 없이는 진행하지 않습니다.

결과 해석 전

실습용 빠른 mesh 결과는 "workflow 성공" 증거입니다. peak spacing, intensity ratio, vacancy fraction 같은 물리 결론은 convergence setting으로 재계산한 뒤 말합니다.

추천 실습 문장. "오늘은 계산을 빨리 끝내기 위해 작은 mesh를 쓰지만, manuscript에서는 unit cell 8^3 , supercell 4^3 mesh로 production spectra를 다시 계산했다. 작은 mesh 결과로 물리 결론을 내리지 않는다."

Sources used for this page

- `oh-my-qe.pptx` screenshots and command examples.
- <https://github.com/material-JH/oh-my-qe> plugin source, command docs, install script, and safety rules.

- manuscript_pr1.pdf and supplement_pr1.pdf O K-edge convergence/method-sensitivity sections.